

FlashThink: An Early Exit Method For Efficient Reasoning

Guochao Jiang^{*†}, Guofeng Quan^{*}, Zepeng Ding, Ziqin Luo, Dixuan Wang, Zheng Hu
Fudan University

Abstract

Large Language Models (LLMs) have shown impressive performance in reasoning tasks. However, LLMs tend to generate excessively long reasoning content, leading to significant computational overhead. Our observations indicate that even on simple problems, LLMs tend to produce unnecessarily lengthy reasoning content, which is against intuitive expectations. Preliminary experiments show that at a certain point during the generation process, the model is already capable of producing the correct solution without completing the full reasoning content. Therefore, we consider that the reasoning process of the model can be exited early to achieve the purpose of efficient reasoning. We introduce a verification model that identifies the exact moment when the model can stop reasoning and still provide the correct answer. Comprehensive experiments on four different benchmarks demonstrate that our proposed method, FlashThink, effectively shortens the reasoning content while preserving the model accuracy. For the Deepseek-R1 and QwQ-32B models, we reduced the length of reasoning content by **77.04%** and **77.47%**, respectively, without reducing the accuracy.

1 Introduction

Recently, researchers have increasingly focused on LLMs (Zhao et al., 2023) and their abilities to solve reasoning tasks (Ke et al., 2025). Using the reasoning chain of thought and reinforcement learning (Havrilla et al., 2024), LLMs have demonstrated strong reasoning ability (Jaech et al., 2024; Guo et al., 2025). For instance, DeepSeek-R1 (Guo et al., 2025) employs Supervised Fine-Tuning (SFT) and Reinforcement Learning (RL), emphasizing a structured thinking template and a rule-based reward mechanism (Shao et al., 2024),

^{*} Equal Contributions.

[†] Corresponding author(s): gcjiang22@m.fudan.edu.cn

Question: Evaluate $(-125)^{4/3}$

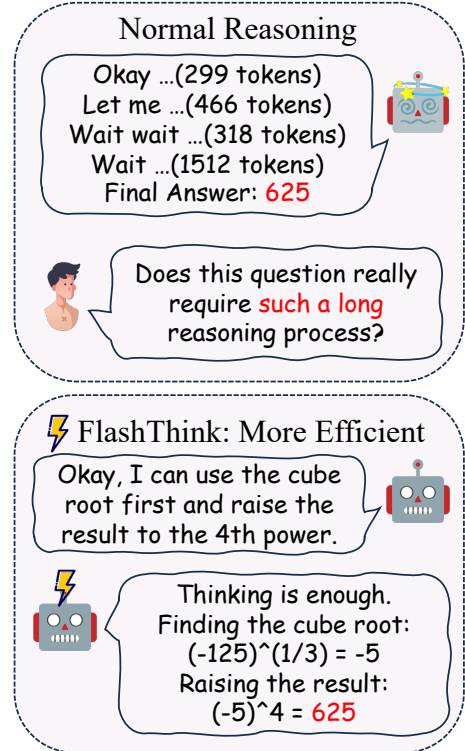


Figure 1: DeepSeek-R1’s reasoning process for an example from the MATH (Hendrycks et al., 2021) benchmark. Normal reasoning will cause multiple thinking changes and multi-angle analysis of this simple question in the reasoning content, which can easily generate many unnecessary tokens. FlashThink significantly reduces the number of tokens in the reasoning content without modifying the original model parameters and without loss of performance, thereby improving the efficiency of model reasoning. The complete content of this example are listed in Appendix A.

which provide precise and explicit feedback signals, boosting the general reasoning capabilities during training.

In the reasoning process, LLMs often produces verbose responses, since extending reasoning content can improve accuracy. This leads to significant

computational overhead and longer inference time cost. Even for simple reasoning questions, LLMs tend to create long thought processes (Chen et al., 2024). This verbosity raises inference costs and delays, limiting the practical applications of reasoning models.

Reducing the length of reasoning content has recently become a major research focus (Sui et al., 2025). One intuitive approach is to add a strategy to limit the reasoning length during training. Some researchers incorporate a length penalty into RL policy optimization to make LLMs generate shorter answers (Team et al., 2025; Yeo et al., 2025; Shen et al., 2025a). Others create datasets with variable-length reasoning content to fine-tune LLMs directly. Another exploration direction is to modify the reasoning paradigm. Continuous thought chain (Hao et al., 2024; Cheng and Van Durme, 2024) treats the last hidden state of the LLM as a “continuous thought” to replace traditional discrete labels. Compressing tokens into latent representations improves accuracy and efficiency by shortening reasoning content.

We introduce a new perspective: large language models frequently overthink for simple questions. During the reasoning process, these models have the capability to produce correct answers after generating just a small portion of the initial reasoning content. When trained on large datasets containing reasoning content, the models tend to generate extended reasoning chains to tackle complex problems. However, they lack the ability to subjectively assess the difficulty of a problem, resulting in the generation of unnecessarily lengthy reasoning even for simple tasks.

As illustrated in Figure 1, the Deepseek-R1 model demonstrates this behavior by frequently revisiting its thought process to formulate the final answer. It often generates paragraphs with tokens such as “Wait” and “Let me check”, highlighting its iterative and redundant approach, even in response to simple questions.

Our work aims to help the LLM identify when to stop thinking and produce the final answer. We use delimiter tokens to divide the model’s reasoning content into chunks. After generating a chunk, a verification model is used to decide whether the thinking process should continue. As shown in Figure 1, once the verification model determines to exit early, the final result will be generated directly without further thinking. We conducted experiments with the DeepSeek-R1 and QwQ series

across four datasets with different difficulty levels. Initially, we used Qwen2.5-7B as the verification model. The results show that FlashThink effectively reduces the length of generated reasoning content (58.19%-94.72% across different datasets) without lowering the acceptance rate. Later, we fine-tuned the verification model, naming it FT². The findings indicate that further optimization of the verification model enhances FT’s ability to reduce content length more effectively.

2 Related Work

Recently, large language models that use reasoning to solve problems often overthink. Effective reasoning (Sui et al., 2025) is mainly achieved through three methods: model-based reasoning, output-based reasoning, and prompt-based reasoning.

Model-based efficient reasoning focus on fine-tuning LLMs to improve their intrinsic ability to reason concisely and efficiently. O1-Pruner (Luo et al., 2025) introduces the Length-Harmonizing Reward, combined with a PPO-style loss, to optimize reasoning LLMs by effectively shortening the CoT length. L1 (Aggarwal and Welleck, 2025) modify the training data with the designated length constraint instruction before launching the policy optimization with pre-trained reasoning LLMs. TokenSkip (Xia et al., 2025) estimates the semantic importance of each reasoning part to the final answer and reduces the reasoning tokens. LearnSkip (Liu et al., 2024) proposes a human-like step-skipping method for generating shorter reasoning steps. However, these methods require fine-tuning of LLMs, incurring unacceptable computational costs.

Reasoning output-based efficient reasoning focus on modifying the output paradigm to enhance the ability of LLMs to reason concisely and efficiently. Coconut (Hao et al., 2024) treats the final-layer hidden states of an LLM as “continuous thought” to replace traditional discrete tokens. CODI (Shen et al., 2025b) leverages a different training process compared to Coconut, which learns the continuous latent CoT via self-distillation. LightThinker (Zhang et al., 2025b) proposes to train LLMs to learn when and how to compress intermediate reasoning steps. However, these methods require modifications to the original LLM reasoning paradigm and the continued training of LLMs, which incurs additional compu-

tational costs.

Input prompts-based efficient reasoning focus on enforcing length constraints or routing LLMs based on the characteristics of input prompts to enable concise and efficient reasoning. CoD (Xu et al., 2025) observes that LLMs often generate excessively verbose reasoning steps, whereas humans typically record only the most essential insights. Sketch-of-Thought (Aytes et al., 2025) leverages routing and prompting to minimize token usage during reasoning. However, these methods often preset token budgets, making it difficult to dynamically adapt to different problems.

Some methods on rejecting the reasoning phase of reasoning models have also greatly improved the reasoning efficiency of LLMs. NoThinking (Ma et al., 2025) method directly optimizes the pass@ k metric as k increases, which generates some outputs independently and aggregates them. Zhang et al. (2025a) uses the probe as a verifier to decide whether to exit reasoning at intermediate answers during inference, reducing the number of inference tokens. However, such methods can only optimize the pass@ k metric but cannot improve the pass@1 of the reasoning models, which greatly reduces the usability of LLMs.

3 Methodology

In recent years, LLMs have shown promising results on many NLP tasks (Jiang et al., 2024b, 2025, 2024a) with reasoning. Most modern reasoning LLMs, like DeepSeek-R1 (Guo et al., 2025), QwQ (Yang et al., 2024), and Phi-4-Reasoning (Abdin et al., 2025), use special tokens <think> and </think> to wrap reasoning content. They output the model’s response after reasoning is completed. This method of using special tokens to sequentially output reasoning and non-reasoning content fits with the autoregressive generation process of LLMs. In this section, we will explain how the FlashThink method works during the generation phase of LLM reasoning models.

3.1 FlashThink

Formally, we define the model input as $x = \{x_1, x_2, \dots, x_m\}$, where x_i is the token of the i -th input and $1 \leq i \leq m$, m is the input length. As mentioned above, the output of the LLM reasoning models usually starts with the reasoning content wrapped in two special tokens <think> and </think>, followed by the non-reasoning con-

John arm wrestles 20 people. He beats 80%. How many people did he lose to?

DeepSeek-R1 Reasoning Content:

Okay, let’s see. So, John arm wrestles 20 people, and he beats 80% of them. The question is, how many people did he lose to? Hmm.

First, I need to figure out how many people he beat, and then subtract that from the total to find out how many he lost to. Because the total number of people is 20, right? So, if he beats 80%, then the number of people he beat is 80% of 20. Let me calculate that. Okay, 80% is the same as 0.8 in decimal form. So, 0.8 multiplied by 20 equals... 16. So he beat 16 people.

Now, if there are 20 people in total, and he beat 16, the number he lost to would be the remainder. That’s 20 minus 16, which is 4. So he lost to 4 people. Let me just make sure that makes sense. 80% of 20 is indeed 16, and 20 minus 16 is 4. Yeah, that checks out.

Wait, but just to double-check, sometimes percentages can confuse me. Let me verify another way. If he beats 80%, then the percentage he lost to would be 100% minus 80%, which is 20%. So 20% of 20 people is 0.2 times 20, which is 4. Yes, same answer. So either way, calculating the number he lost to directly by subtracting the 80% from 100% and taking the percentage of 20, or taking 80% of 20 and subtracting from the total, both methods lead to 4.

I think that’s solid. So the answer should be 4. He lost to 4 people out of the 20 he arm wrestled. Can’t see a mistake here. Let me make sure I didn’t misread the problem. So, the question says he arm wrestles 20 people, beats 80%, how many did he lose to? Yep, that’s straightforward. 80% beaten, the rest lost. 20 - (0.8*20)=4. So yeah, it’s 4.

Another angle: If you think, 10% of 20 is 2, so 80% is 8 times that, which is 16. Then 20-16=4. Yep, same thing. Okay, this is too straightforward, but maybe there’s a trick? Hmm, could the question be implying something else?

Wait, maybe it’s about interpretation. “Beats 80%” – does that mean 80% of his matches or 80% of the people? The problem says he arm wrestles 20 people. So, “beats 80%” of those people, I would assume. So each person he wrestles, he either beats or loses to. If there are 20 people total, he must have 20 matches (assuming one match per person). So in those 20 matches, he beats 80%, which is 16, loses 4. Yeah, that makes sense.

But another thought, could there be draws? But the problem doesn’t mention draws or ties. It just says he beats 80%, so the remaining 20% would be losses. All right, so in a standard arm wrestling match, it’s either a win or a loss, so no draw considered here unless specified. Therefore, the answer is 4.

Another possible confusion point: Is “beats 80%” referring to 80% success rate overall or something else? For example, beats 80% of the time? But he’s wrestling 20 people, so that would be cumulative. So again, 80% of 20 people is 16, which he beats, the rest 20% of 20 people is 4, which he lost to. I think I’ve thought it through all possible angles. The answer is 4, and there’s no ambiguity here. Yes, if he had 20 matches, winning 80% of them means 16 wins and 4 losses.

****Final Answer****
He lost to 4 people.

Figure 2: A case from GSM8K, generated by DeepSeek-R1. Note that the reasoning content contains many thinking changes, but the reasoning content has basically solved the problem before the first thinking change.

tent of the LLM reasoning models. In order to split the reasoning content, we predefine a set $S = \{s_1, s_2, \dots, s_k\}$ of delimiter tokens, where $s_i, 1 \leq i \leq k$ is the i -th separation token, k is size of the delimiter tokens set. According to the delimiter tokens set S , we can split the model reasoning content r into reasoning chunks $c_1, c_2, \dots, c_{|r|}$, where $|r|$ is the reasoning chunks count.

We denote the output of the model’s non-reasoning content as $y = \{y_1, y_2, \dots, y_n\}$, where $y_i, 1 \leq i \leq n$ is the token of the i -th non-reasoning content, n is the non-reasoning content length. The whole generation process is recorded as

$$\begin{aligned} y &= \text{LLM}_\theta(x|r) \\ &= \text{LLM}_\theta(x|c_1, c_2, \dots, c_{|r|}), \end{aligned} \quad (1)$$

where θ is the LLM parameters.

Not all reasoning content is necessary. These reasoning contents often contain more “aha moments”

(Guo et al., 2025) and changes in thinking. Reasoning can derive interim results for input questions early on. These interim results are often correct for simple questions. Therefore, ending the reasoning phase early can significantly reduce the number of tokens and improve LLM inference efficiency. As shown in Figure 2, the problem is relatively easy, and the answer is $20 \times (1 - 80\%) = 4$. However, in the DeepSeek-R1 model’s reasoning content, there are many descriptions of thinking changes like "Wait" and "Another angle" leading to a lot of unnecessary content. This greatly reduces the efficiency of the reasoning process. In fact, the reasoning content before "Wait" is enough to solve the question.

The FlashThink method relies on previous observations. It uses an existing verification model to determine if the current reasoning content is enough to solve the problem. This helps decide whether to exit the reasoning phase early and boost its effectiveness. The verification model is represented as π . For a given input x and the current reasoning content chunks r' , it produces a boolean value $\pi(x|r')$. This value indicates whether to end the reasoning phase early or create the next reasoning content chunk. The FlashThink algorithm process is shown in Algorithm 1.

Algorithm 1: FlashThink Algorithm

Require: Reasoning Model LLM_θ , Verification Model π , Input x , Reasoning content r , Output y , Delimiter set S .

- 1: Initialize $r \leftarrow \square$
- 2: Initialize finished $\leftarrow \text{false}$
- 3: Initialize $y \leftarrow \text{None}$
- 4: **while not finished do**
- 5: Generate next token t from $\text{LLM}_\theta(x|r)$
- 6: $r \leftarrow [r; t]$
- 7: **if** $t \in S$ **then**
- 8: decision $\leftarrow \pi(x|r)$
- 9: **if** decision is **true** **then**
- 10: $y \leftarrow \text{LLM}_\theta(x|r)$
- 11: finished $\leftarrow \text{true}$
- 12: **end if**
- 13: **end if**
- 14: **end while**
- 15: **return** r, y

3.2 FT²: Fine-Tuning FlashThink

As mentioned earlier, the FlashThink method needs the verification model π to assess how well the input prompt matches the part of the reasoning content. This requires the verification model to have good judgment ability. However, as the input data and reasoning model change, the verification model with fixed parameters struggles to adapt to the shifts in input distribution caused by these changes.

In order to better cope with shifts in input distribution, we further propose a method for fine-tuning the verification model, FT². FT² targets specific input data distribution and specific LLM reasoning ability, and improves the FlashThink capability of the verification model for this specific data and model combination through fine-tuning, so as to further reduce the loss of model accuracy and improve the efficiency of the model in the reasoning phase.

For the training data of the FT² method, we mainly construct positive and negative samples by whether they support the results of the FlashThink method. Specifically, for the input x and the reasoning content chunks $c_1, c_2, \dots, c_{|r|}$, we first use the reasoning model to generate the results $y^{(1)}, y^{(2)}, \dots, y^{(|r|)}$ for different reasoning content chunk order combinations, that is,

$$y^{(i)} = \text{LLM}_\theta(x|c_1, c_2, \dots, c_i), 1 \leq i \leq |r|. \quad (2)$$

If the score of the result $y^{(i)}$ output by FlashThink is consistent with the score of the original output y or $y^{(i)}$ is positive result for the question, the data $(x, c_{1:i}, y^{(i)})$ is considered to be a positive sample, otherwise it is a negative sample. Theoretically, the verification model can be any binary classification model. This paper uses LLMs as the verification model, and the prompts used are shown in the Appendix B. Based on these positive and negative samples, we can fine-tune the verification model to obtain a verification model that adjusts the input data distribution.

4 Experiments

In this section, we will show the results of FlashThink method on multiple benchmarks, covering multiple reasoning models. We also discuss the impact of different models as verification models and the effect of FT² method. Multiple case studies will also be included.

Benchmark	Train Sample #	Test Sample #
GSM8K	7,473	1,319
MATH	7,500	5,000
GPQA Diamond	994	198
DROP	77,409	9,536

Table 1: Experimental benchmarks’ statistics

4.1 Experimental setup

Benchmarks. In order to fully discuss the reasoning ability of the reasoning model, we select some mathematical benchmark datasets and knowledge and reasoning benchmark datasets. For math test benchmarks, we select GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021). For knowledge and reasoning benchmarks, we select GPQA Diamond (Rein et al., 2024) and DROP (Dua et al., 2019). Among these benchmarks, GSM8K, MATH, and DROP have corresponding datasets divided into train and test. For the GPQA dataset, we choose GPQA Diamond as the test set and GPQA Main and GPQA Extended as the train set. In order to ensure the accuracy of experimental tests, we do not downsample the dataset in terms of quantity. Table 1 gives some basic information on the train set and the test set of these benchmarks.

Reasoning Models. We use DeepSeek-R1¹, QwQ-32B², DeepSeek-R1-Distill-Llama-70B³ and DeepSeek-R1-Distill-Qwen-32B⁴ as the main reasoning models. To ensure strong reasoning capabilities, we avoided models with fewer parameters.

Verification Models. In the main experiment, we use Qwen2.5-7B-Instruct⁵ as the verification model. In the verification model experiment, we expand the scope of verification models to verify the impact of different verification models on the FlashThink method. Specifically, we cover the Qwen2.5 series (Yang et al., 2024), Llama-3.1-8B-Instruct⁶, Mistral-7B-Instruct-v0.3⁷, and Qwen3 series (Yang et al., 2025).

Evaluation and Metrics. For benchmark result

¹<https://huggingface.co/deepseek-ai/DeepSeek-R1>

²<https://huggingface.co/Qwen/QwQ-32B>

³<https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-70B>

⁴<https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B>

⁵<https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>

⁶<https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>

⁷<https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>

evaluation, we adopt the evaluation prompts from the simple-evals framework⁸, in which Math-Verify framework⁹ is introduced for mathematical evaluation. GSM8K, MATH, and GPQA Diamond use the Accuracy metric, and DROP uses the F1 score. All the maximum values of the scores shown are normalized to 100.00 for comparison. DROP uses the 3-shot setting, and other benchmarks use the 0-shot setting. We set the maximum generation length to 32,768 tokens for the models. We report the pass@1 (Chen et al., 2021) results using a temperature of 0.7. For the inference deployment of these models, we use the SGLang framework (Zheng et al., 2024a) for efficient inference to reduce time cost. In order to compare the efficiency of the methods for efficient reasoning, we define the reduction in the number of tokens in the current reasoning content relative to the number of tokens in the original reasoning content as Efficiency.

Implementation details. We conduct our experiments on $8 \times$ NVIDIA H20 GPUs. For fine-tuning of the FT² method, we use fine-tuning method for all parameters, training for 3 epochs, with a maximum length of 16,384, a batch size of 16, an AdamW (Loshchilov and Hutter, 2019) optimizer, and a learning rate of 1e-5. For fine-tuning of models, we perform ZeRO-3 (Rajbhandari et al., 2020) training method based on the LlamaFactory (Zheng et al., 2024b). FlashAttention-2 (Dao, 2024) is used to accelerate the attention calculation stage. Liger kernel (Hsu et al., 2024) is used throughout the process to reduce memory consumption and speed up calculations.

4.2 Main Results

Table 2 shows the main experimental results, where the verification model uses Qwen2.5-7B-Instruct, and the reasoning model uses two main reasoning models DeepSeek-R1 and QwQ-32B and two distilled open-source models DeepSeek-R1-Distill-Llama-70B and DeepSeek-R1-Distill-Qwen-32B. The experimental results show the results of normal reasoning generation and use of FlashThink on four benchmark datasets GSM8K, MATH, GPQA Diamond and DROP, including accuracy scores and efficiency metrics of the FlashThink method.

The comprehensive evaluation displayed in Table 2 underscores the consistency of accuracy levels across different reasoning models when enhanced with the FlashThink method. By examining the

⁸<https://github.com/openai/simple-evals>

⁹<https://github.com/huggingface/Math-Verify>

Reasoning Model	GSM8K		MATH		GPQA Diamond		DROP		Average	
	Acc.	Efficiency	Acc.	Efficiency	Acc.	Efficiency	F1	Efficiency	Score	Efficiency
DeepSeek-R1	92.87	-	96.76	-	67.68	-	90.70	-	87.00	-
w/ FlashThink	93.99	90.91%	97.24	94.72%	67.66	64.35%	89.70	58.19%	87.15	77.04%
QwQ-32B	92.69	-	95.20	-	58.61	-	87.73	-	83.56	-
w/ FlashThink	92.65	89.60%	95.90	93.55%	58.08	65.32%	88.84	61.39%	83.87	77.47%
R1-Distill-Llama-70B	85.57	-	93.30	-	61.64	-	82.11	-	80.66	-
w/ FlashThink	87.26	75.73%	94.80	84.35%	53.53	60.23%	83.30	59.99%	79.72	70.08%
R1-Distill-Qwen-32B	86.40	-	93.44	-	57.60	-	80.46	-	79.48	-
w/ FlashThink	88.32	76.35%	93.72	69.62%	56.06	32.13%	80.73	69.21%	79.71	61.83%

Table 2: The performance comparison of four reasoning models (DeepSeek-R1, QwQ-32B, R1-Distill-Llama-70B and R1-Distill-Qwen-32B) on different benchmarks (GSM8K, MATH, GPQA Diamond, DROP), including Score (Accuracy or F1) and Reasoning Efficiency improvement (Efficiency). At the same time, the results also show the performance changes of each model after combining FlashThink method. Among them, the experiments with advantages in score will be **bolded** in each group.

average score results, it is evident that FlashThink integration preserves, and in certain instances improves, the overall score of the models. For instance, DeepSeek-R1’s average score improved marginally from 87.00 to 87.15 after integrating FlashThink, demonstrating no loss in accuracy. Similarly, QwQ-32B maintained its average score of 83.56, slightly enhancing to 83.87 with FlashThink. In the case of R1-Distill-Llama-70B, the average score marginally dropped from 80.66 to 79.72, though this reduction remains within an acceptable range, indicating minimal impact. The R1-Distill-Qwen-32B model’s average score remained essentially stable, slightly adjusting from 79.48 to 79.71. These average results across diverse data sets substantiate the claim that FlashThink integration results in accuracy retention, highlighting its capability to optimize performance without detriment to accuracy across multiple reasoning tasks.

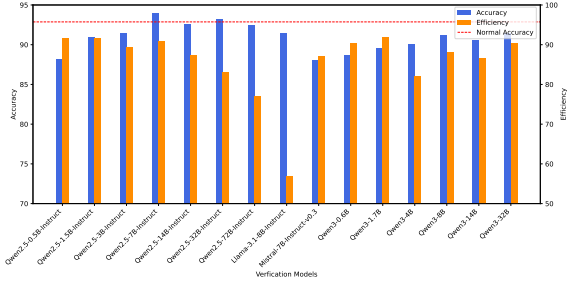
The integration of FlashThink into existing reasoning models not only maintains accuracy but also significantly enhances the efficiency of content generation during reasoning. Examining the average Efficiency metrics, DeepSeek-R1 with FlashThink achieves a substantial increase in efficiency to 77.04%, demonstrating a marked improvement in the generation process without sacrificing performance. Similarly, the efficiency of QwQ-32B increases to 77.47% with FlashThink, indicating a streamlined reasoning process while preserving its accuracy level. The R1-Distill-Llama-70B and R1-Distill-Qwen-32B models also benefit from FlashThink integration, with efficiency gains to 70.08% and 61.83%, respectively. These improvements reflect a noteworthy reduction in the computational resources and time required for generating reasoning content, underscoring FlashThink’s capabil-

ity to enhance inference efficiency across various models and datasets. Consequently, FlashThink emerges as a powerful tool for optimizing reasoning processes, achieving efficiency gains while ensuring the accuracy remains intact.

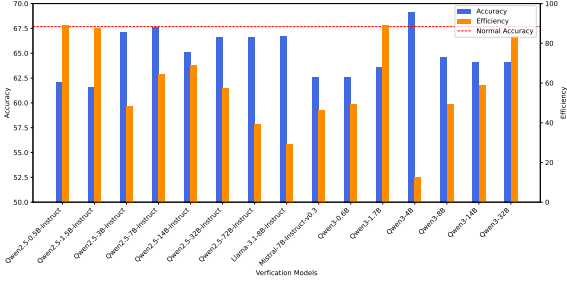
While FlashThink integration consistently enhances efficiency across different reasoning models, the extent of these enhancements varies depending on the specific model and dataset. For instance, DeepSeek-R1 experiences substantial efficiency improvements, achieving an average efficiency increase to 77.04%, with particularly notable gains on the MATH dataset at 94.72%. In contrast, the same model shows less pronounced efficiency improvements on the GPQA Diamond dataset. Similarly, R1-Distill-Qwen-32B exhibits an overall efficiency rise to 61.83%, yet its efficiency gain on the GPQA Diamond dataset remains relatively modest at 32.13%. This variability suggests that FlashThink’s optimization effectiveness is influenced by the intrinsic properties of the models and the complexity or nature of the datasets. Such differential impacts highlight the importance of context-specific evaluations to fully leverage FlashThink’s potential, pointing to opportunities for tailored optimizations that take into account the specific characteristics of each application scenario.

4.3 Verification Model Experiment

Figure 3 shows the performance and efficiency comparison of FlashThink when using different models as verification models for the inference model DeepSeek-R1 on the two reasoning benchmarks GSM8K and GPQA Diamond. For the relatively easy benchmark GSM8K, in the Qwen-2.5 series models, as the parameter amount scales up,



(a) The performance and efficiency with different verification models on GSM8K



(b) The performance and efficiency with different verification models on GPQA Diamond

Figure 3: Verification Model Experiments.

Reasoning Model	GSM8K		GPQA Diamond	
	Acc.	Efficiency	Acc.	Efficiency
DeepSeek-R1	92.87	-	67.68	-
w/ FlashThink	93.99	90.81%	67.66	64.35%
w/ FT ²	93.57	93.66%	68.75	70.33%
QwQ-32B	92.69	-	58.61	-
w/ FlashThink	92.65	89.60%	58.08	65.32%
w/ FT ²	94.78	92.72%	62.73	75.64%

Table 3: The comparative performance of different reasoning models evaluated on the GSM8K and GPQA Diamond datasets using FlashThink or FT².

FlashThink’s performance loss will gradually decrease, and the efficiency will also decrease. In the Qwen-3 series, FlashThink’s performance is similar, while the efficiency fluctuates at a relative level. At the order of 7B, the two metrics of Qwen2.5-7B-Instruct are relatively better under the four verification models. For the relatively difficult benchmark GPQA Diamond, similar trends also exist in the Qwen-2.5 series and Qwen-3 series. Therefore, the choice of verification model has a great impact on the performance loss and efficiency of FlashThink. In contrast, Qwen2.5-7B-Instruct is a better choice, which is also verified in the main results in Table 2.

4.4 FT² Performance Experiment

The data presented in Table 3 highlight the significant improvements achieved by employing the FT² method over the baseline models and those enhanced with FlashThink. Notably, FT² consistently enhances both accuracy and efficiency across various reasoning models and datasets. For instance, the DeepSeek-R1 model’s accuracy increased from 93.99 with FlashThink to 93.57 with FT² on the GSM8K dataset, while efficiency improved from 90.81% to 93.66%. Similarly, the QwQ-32B model showed remarkable improvements in accuracy and efficiency on the GPQA Diamond dataset, from 58.08 accuracy with FlashThink to 62.73 with FT², and corresponding efficiency improvements from 65.32% to 75.64%. These results conclusively demonstrate that FT² not only refines model precision but also substantially enhances the efficiency of content generation during reasoning tasks, outperforming the FlashThink enhancement across diverse datasets and models. This is due to the fact that FT² fine-tunes the verification model to better learn the current input distribution for the current reasoning model and the current evaluation benchmark, and provides better verification capabilities for reasoning model based on pre-planned supervision signals.

4.5 Case Study

In the above, we present some examples of complicating simple problems due to frequent thinking changes in reasoning models, resulting in the generation of too long reasoning content and thus reducing the efficiency of problem solving. In the case study we will give more examples and further analyze how FlashThink can bring efficiency improvements. See Appendix A for more cases.

As shown in below, the DeepSeek-R1 model’s response to the question $2^8 = 4^x$ is overly verbose and complicated, expounding on multiple unnecessary methods and hypothetical scenarios. This complexity is disproportionate to the simplicity of the question, which can be solved directly by recognizing that $4 = 2^2$ and equating the exponents. In contrast, the FlashThink method efficiently provides a concise and straightforward solution, focusing on expressing both sides with the same base to quickly find that $x = 4$. This highlights FlashThink’s advantage in offering clarity and brevity, essential for effective reasoning for reasoning LLMs.

For the other cases in Appendix A, FlashThink

ends the reasoning phase quickly, directly generates a concise and intuitive solution, and gets the correct answer. However, when reasoning content is normally generated, it often has to go through many changes in thinking and multi-angle solution verification, which isn't very helpful for solving simple questions concisely.

If $2^8 = 4^x$, what is the value of x ?

DeepSeek-R1 (1850 tokens):

<think>

Okay, so I have this equation here: 2 to the power of 8 equals 4 to the power of x . I need to find the value of x . Let me start by writing it out again to get a clearer picture: $2^8 = 4^x$. Hmm, right. Both sides of the equation are powers of 2 and 4. Wait, but 4 is a power of 2, isn't it? Yeah, 4 is 2 squared. So, maybe I can express both sides with the same base. That might make it easier to compare the exponents.

Let me try rewriting 4 as 2^2 . So substituting that into the equation, the right-hand side, which is 4^x , becomes $(2^2)^x$. Now, recall the exponentiation rule: $(a^b)^c = a^{bc}$. So applying that here, $(2^2)^x$ becomes 2^{2x} . So now the equation is $2^8 = 2^{2x}$.

Now that both sides of the equation have the same base, 2, I can set the exponents equal to each other. Because if $a^b = a^c$, then $b = c$, right? As long as a is not 0 or 1, which it isn't here since a is 2. So that must mean $8 = 2x$. Okay, so solving for x should be straightforward now. Dividing both sides by 2 gives $x = 8/2 = 4$. So x is 4.

Wait, let me check if that makes sense. Let's verify the original equation with $x = 4$. So 4^4 is $4 * 4 * 4 * 4$. 4 squared is 16, 16 squared is 256. So 4^4 is 256. On the left side, 2^8 is $2 * 2 * 2 * 2 * 2 * 2 * 2 * 2$. 2^3 is 8, 2^4 is 16, 2^5 is 32, up to 2^8 , which would be 256. Yep, both sides are 256. So $x = 4$ works. Hmm.

Alternatively, I could have approached this using logarithms, maybe, to see if it gives the same answer. Taking the logarithm of both sides. Let's try that approach just for confirmation. Starting from $2^8 = 4^x$. Take the natural logarithm of both sides: $\ln(2^8) = \ln(4^x)$. Using the logarithm power rule, which says $\ln(a^b) = b \ln(a)$. So then that becomes $8 * \ln(2) = x * \ln(4)$. And solving for x gives $x = \frac{8 \ln 2}{\ln 4}$. But since 4 is 2 squared, $\ln 4$ is $\ln(2^2) = 2 \ln 2$. Substituting this into the denominator, we get $x = \frac{8 \ln 2}{2 \ln 2} = \frac{8}{2} = 4$. Yeah, same result. So even if I use logarithms, it still gives $x = 4$. Good, that confirms it.

Alternatively, maybe someone might think to convert both sides to base 4. Let's see if that works too. If you convert 2^8 to base 4. Since 2 is the square root of 4, which is $4^{\frac{1}{2}}$. So 2^8 is $(4^{\frac{1}{2}})^8$. Then again, using the exponentiation rule: $(a^b)^c = a^{bc}$. So that becomes $4^{\frac{1}{2} * 8} = 4^4$. Therefore, the equation becomes $4^4 = 4^x$, which implies $x = 4$. That's the same answer too. So regardless of the method, base conversion or logarithms, x equals 4. So that must be the correct answer.

Wait, but let me make sure there's no alternative interpretation. Sometimes, equations can have multiple solutions, but here, given that both sides are exponential functions with base greater than 1, they are strictly increasing, so the equation should have exactly one solution. Since we found $x = 4$, that's the only solution. Also, since the exponents must be included for that. So considering that 2 and 4 are positive real numbers not equal to 1, the one-to-one property of exponential functions holds here. Therefore, x is uniquely determined, and that's 4.

Alternatively, just for robustness, suppose someone made a mistake in thinking 4 is $2 + 2$. Then maybe they try some arithmetic addition of exponents. But that's a common mistake students sometimes make. For instance, maybe confusing $2^8 * 2^2 = 2^{8+2} = 2^{10}$, which is correct. But here it's 4^x , which is $(2^2)^x$. So mixing up operations could lead someone to add instead of multiply, but in this case, the idea is to recognize that since 4 is a power of 2, you can rewrite the equation with the same base. Then, since you have $(2^2)^x$ and 2^8 , easy to equate exponents.

Another way I can think of is prime factorization. Both 2 and 4 are already prime factors of themselves, 4 being 2 squared. So, only prime involved here is 2. So, right-hand side is $4^x = (2^2)^x = 2^{2x}$, and left-hand side is 2^8 , so set $2x = 8$, so $x = 4$.

Alternatively, in more concrete terms, for those who need numerical evaluation, maybe compute 2^8 first. 2^2 is 4, 2^3 is 8, up to 2^8 which is 256. Similarly, 4^1 is 4, 4^2 is 16, 4^3 is 64, 4^4 is 256. Thus, seeing when 4^x reaches 256. So that's at $x = 4$. Therefore, same answer.

Alternatively, trial and error. Maybe plug in $x = 3$: 4^3 is 64; too small. $x = 5$: 4^5 is 1024, too big. 4^4 is 256. 2^8 is 256, so they match. Therefore, $x = 4$. So all approaches are in agreement.

This question is testing the knowledge of exponent rules, specifically recognizing that four is a power of two and thus converting both sides of the equation to the same base to equate the exponents. The correct answer is 4.

Wait, just to make sure, are there any exceptions here I need to consider? For example, what if the base was negative? But 2 and 4 are positive, so no issues. What about if x was a fraction or negative? But in this case, since 2^8 is positive, and 4^x is always positive, and because exponentials are injective over the positive reals, so x must be 4.

Alternatively, suppose someone considers complex numbers, but that's beyond the scope here. The problem is presented with integers, so unless it's indicated

otherwise, x is expected to be real, particularly an integer here. So x is 4. Conclusion: The value of x is 4.

****Final Answer****

The value of x is 4.

</think>

Given the equation $2^8 = 4^x$, we need to find the value of x .

First, express 4 as a power of 2: $4 = 2^2$. Substituting this into the equation, we get:

$$4^x = (2^2)^x$$

Using the exponentiation rule $(a^b)^c = a^{bc}$, we can rewrite the right-hand side:

$$(2^2)^x = 2^{2x}$$

Now, the equation becomes:

$$2^8 = 2^{2x}$$

Since the bases are the same, we can set the exponents equal to each other:

$$8 = 2x$$

Solving for x , we divide both sides by 2:

$$x = \frac{8}{2} = 4$$

Verification:

- The left-hand side $2^8 = 256$

- The right-hand side $4^4 = 256$

Both sides are equal, confirming the solution is correct.

Thus, the value of x is 4.

FlashThink (137 tokens):

<think>

Okay, so I have this equation here: 2 to the power of 8 equals 4 to the power of x . I need to find the value of x . Let me start by writing it out again to get a clearer picture: $2^8 = 4^x$. Hmm, right. Both sides of the equation are powers of 2 and 4. Wait, but 4 is a power of 2, isn't it? Yeah, 4 is 2 squared. So, maybe I can express both sides with the same base. That might make it easier to compare the exponents.

</think>

To solve the equation $2^8 = 4^x$, express both sides with the same base. Since $4 = 2^2$, substitute this into the equation:

$$4^x = (2^2)^x = 2^{2x}$$

Now, the equation becomes:

$$2^8 = 2^{2x}$$

Since the bases are equal, the exponents must be equal:

$$8 = 2x$$

Solving for x gives:

$$x = \frac{8}{2} = 4$$

****Final Answer****

The value of x is 4.

5 Conclusion

In this paper, we introduce the FlashThink method to address the issue of existing reasoning models overthinking for simple questions. FlashThink uses a verification model to early exit the reasoning phase without changing the original reasoning model's parameters or the autoregressive model generation paradigm. This approach keeps the performance of the model while making the reasoning phase more efficient, significantly cutting down on costs. Our experiments on four reasoning benchmarks illustrate the advantages of FlashThink over the normal reasoning method.

Limitations

We only discuss the scenario when the verification model is a generative model of a large language model, and do not cover the verification

model to multiple model architectures, such as MLP, Encoder-only Transformers, etc. This limitation highlights the potential for future work to explore different ways to understand efficient reasoning.

Ethics Statement

We hereby declare that all authors of this article are aware of and adhere to the provided ACL Code of Ethics and honor the code of conduct.

Use of Human Annotations Human annotations are only used in methodological research at the beginning of the work, to assist in analyzing the feasibility of the proposed solution. Annotators consented to the use of data for research purposes. We ensure that the privacy of all annotators is protected throughout the annotation process, and all of them are adequately paid according to local standards. Human annotations are not applied during the evaluation of our method.

Risks In this paper, all datasets are obtained from official sources. The datasets adopted have been anonymized and do not contain offensive information. However, we cannot guarantee that the datasets do not contain socially harmful or toxic language.

References

- Marah Abdin, Sahaj Agarwal, Ahmed Awadallah, Vidhisha Balachandran, Harkirat Behl, Lingjiao Chen, Gustavo de Rosa, Suriya Gunasekar, Mojan Javaheripi, Neel Joshi, and 1 others. 2025. Phi-4-reasoning technical report. *arXiv preprint arXiv:2504.21318*.
- Pranjal Aggarwal and Sean Welleck. 2025. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*.
- Simon A Aytes, Jinheon Baek, and Sung Ju Hwang. 2025. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, and 1 others. 2024. Do not think that much for $2+3=?$ on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187*.
- Jeffrey Cheng and Benjamin Van Durme. 2024. Compressed chain of thought: Efficient reasoning through dense representations. *arXiv preprint arXiv:2412.13171*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Tri Dao. 2024. [Flashattention-2: Faster attention with better parallelism and work partitioning](#). In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. [DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 2368–2378. Association for Computational Linguistics.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*.
- Alex Havrilla, Yuqing Du, Sharath Chandra Raparthy, Christoforos Nalmpantis, Jane Dwivedi-Yu, Maksym Zhuravinskyi, Eric Hambro, Sainbayar Sukhbaatar, and Roberta Raileanu. 2024. Teaching large language models to reason with reinforcement learning. *arXiv preprint arXiv:2403.04642*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. [Measuring mathematical problem solving with the MATH dataset](#). In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.
- Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. 2024. Liger kernel: Efficient triton kernels for llm training. *arXiv preprint arXiv:2410.10989*.

- Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, and 1 others. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720*.
- Guochao Jiang, Zepeng Ding, Yuchen Shi, and Deqing Yang. 2024a. P-icl: point in-context learning for named entity recognition with large language models. *arXiv preprint arXiv:2405.04960*.
- Guochao Jiang, Ziqin Luo, Chengwei Hu, Zepeng Ding, and Deqing Yang. 2025. [Mitigating out-of-entity errors in named entity recognition: A sentence-level strategy](#). In *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, pages 7754–7765. Association for Computational Linguistics.
- Guochao Jiang, Ziqin Luo, Yuchen Shi, Dixuan Wang, Jiaqing Liang, and Deqing Yang. 2024b. [Toner: Type-oriented named entity recognition with generative language model](#). In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation, LREC/COLING 2024, 20-25 May, 2024, Torino, Italy*, pages 16251–16262. ELRA and ICCL.
- Zixuan Ke, Fangkai Jiao, Yifei Ming, Xuan-Phi Nguyen, Austin Xu, Do Xuan Long, Minzhi Li, Chengwei Qin, Peifeng Wang, Silvio Savarese, and 1 others. 2025. A survey of frontiers in llm reasoning: Inference scaling, learning to reason, and agentic systems. *arXiv preprint arXiv:2504.09037*.
- Tengxiao Liu, Qipeng Guo, Xiangkun Hu, Cheng Jiayang, Yue Zhang, Xipeng Qiu, and Zheng Zhang. 2024. [Can language models learn to skip steps?](#) In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Ilya Loshchilov and Frank Hutter. 2019. [Decoupled weight decay regularization](#). In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net.
- Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. 2025. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*.
- Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. 2025. Reasoning models can be effective without thinking. *arXiv preprint arXiv:2504.09858*.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. [Zero: memory optimizations toward training trillion parameter models](#). In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, page 20. IEEE/ACM.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. [GPQA: A graduate-level google-proof q&a benchmark](#). In *First Conference on Language Modeling*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, and 1 others. 2024. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, and Shiguo Lian. 2025a. Dast: Difficulty-adaptive slow-thinking for large reasoning models. *arXiv preprint arXiv:2503.04472*.
- Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. 2025b. Codi: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*.
- Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, and 1 others. 2025. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, and 1 others. 2025. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*.
- Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. 2025. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*.
- Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. 2025. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, and 1 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. 2025. Demystifying long chain-of-thought reasoning in llms. *arXiv preprint arXiv:2502.03373*.

Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. 2025a. Reasoning models know when they’re right: Probing hidden states for self-verification. *arXiv preprint arXiv:2504.05419*.

Jintian Zhang, Yuqi Zhu, Mengshu Sun, Yujie Luo, Shuofei Qiao, Lun Du, Da Zheng, Huajun Chen, and Ningyu Zhang. 2025b. Lightthinker: Thinking step-by-step compression. *arXiv preprint arXiv:2502.15589*.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, and 1 others. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024a. [Sglang: Efficient execution of structured language model programs](#). In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyang Luo. 2024b. [LlamaFactory: Unified efficient fine-tuning of 100+ language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 400–410, Bangkok, Thailand. Association for Computational Linguistics.

A Reasoning Examples

We present the full response and FlashThink response of the example shown in Figure 1 and other benchmarks’ cases.

GSM8K Case 1

Question:

A fruit vendor bought 50 watermelons for \$80. He sold all of them at a profit of 25%. How much was each watermelon sold?

DeepSeek-R1 (1331 tokens):

<think>

Okay, let’s see. So the problem is about a fruit vendor who bought 50 watermelons for \$80, and then sold all of them at a 25% profit. The question is asking how much each watermelon was sold for. Hmm, let’s break this down step by step.

First, I need to figure out the total cost price for the watermelons. The vendor bought 50 watermelons for \$80. That means the cost per watermelon would be the total cost divided by the number of watermelons. Let me write that down: Cost per watermelon = Total cost / Number of watermelons. So that’s \$80 divided by 50. Let me do that calculation. Hmm, 80 divided by 50. Well, 50 goes into 80 once, which is 50, and then there’s 30 left. 30 divided by 50 is 0.6. So 1 + 0.6 is 1.6. So each watermelon cost \$1.60. But wait, maybe I should write that as a decimal to be precise. Yes, so \$1.60 per watermelon.

But actually, maybe I should do this division properly. 80 divided by 50. Let’s think in terms of dollars. If 50 watermelons cost \$80, then 1 watermelon costs 80/50 dollars. Which simplifies to 8/5, which is 1.6. So yeah, \$1.60 per watermelon. Okay, that seems right.

Now, he sold them at a 25% profit. So the selling price is the cost price plus 25% of the cost price. Alternatively, selling price is 125% of the cost price. Let’s confirm that. If he made a 25% profit, that means the selling price is 100% + 25% = 125% of the cost.

So we can calculate the total selling price first, and then find the price per watermelon, or calculate the selling price per watermelon directly. Maybe both ways to check.

Let me try calculating the total selling price first. The total cost is \$80. 25% profit on \$80. So 25% of 80 is 0.25 * 80 = 20. Therefore, total selling price is 80 + 20 = \$100. So he sold all 50 watermelons for \$100. Then, the selling price per watermelon is 100 divided by 50, which is \$2.00. So that would mean each watermelon was sold for \$2.

But let me check the other way too, just to make sure. If he’s selling each watermelon at a 25% profit, then the selling price per watermelon is 1.25 times the cost price. The cost price per watermelon is \$1.60. So 1.25 * 1.60. Let’s compute that.

1.25 * 1.60. Hmm. 1 * 1.60 is 1.60, and 0.25 * 1.60 is 0.40. So adding them together, 1.60 + 0.40 = \$2.00. Okay, so that makes sense. So each watermelon was sold for \$2.00. That matches the total selling price calculation. So both methods give the same result, \$2.00 per watermelon.

Therefore, the answer should be \$2.00. Just to recap: 50 watermelons cost \$80, so each is \$1.60. 25% profit on each would add \$0.40 (since 1.60 * 0.25 = 0.40), resulting in a selling price of \$2.00. Alternatively, total cost \$80, total selling price \$100, divided by 50 gives \$2.00 each. Both confirm the same answer.

I don’t think there are any other factors here. The problem states a straight profit percentage on the cost, so we don’t need to consider anything else like taxes or discounts or anything. Seems straightforward. Could the percentage be on the selling price instead? Hmm, but usually when it says profit of 25%, unless specified otherwise, it’s on the cost price. If it’s on the selling price, the answer would be different. But since both methods assuming profit on cost give the same answer, and the question doesn’t specify otherwise, I think this is correct. Wait, let’s just double-check the percentage. Profit percentage on cost: Profit / Cost * 100. If profit is \$20 on a cost of \$80, that’s 25%, which matches. So that’s correct. If we were to calculate based on selling price, Profit / Selling Price * 100 would be 20/100 * 100 = 20%. But since the question states a profit of 25%, that’s in line with cost, not selling price. So we’re good.

Yes, all right. So I think the answer is \$2.00 per watermelon.

Final Answer

The vendor sold each watermelon for 2 dollars.

</think>

The fruit vendor bought 50 watermelons for \$80. To find the cost price per watermelon, we divide the total cost by the number of watermelons:

Cost per watermelon = $\frac{80}{50} = 1.60$

The vendor sold all watermelons at a 25% profit. The total selling price is calculated by adding 25% of the total cost to the total cost:

Profit = $0.25 \times 80 = 20$

Total selling price = $80 + 20 = 100$

To find the selling price per watermelon, we divide the total selling price by the number of watermelons:

Selling price per watermelon = $\frac{100}{50} = 2$

We can also verify this by calculating the selling price per watermelon directly.

The selling price per watermelon is 125% of the cost price per watermelon:

Selling price per watermelon = $1.25 \times 1.60 = 2.00$

Both methods confirm that the selling price per watermelon is \$2.00.

2

FlashThink (151 tokens):

<think>

Okay, let’s see. So the problem is about a fruit vendor who bought 50 watermelons for \$80, and then sold all of them at a 25% profit. The question is asking how much each watermelon was sold for. Hmm, let’s break this down step by step.

</think>

First, determine the total cost price of the watermelons, which is \$80 for 50 watermelons. To find the cost price per watermelon, divide the total cost by the number of watermelons:

$$\text{Cost per watermelon} = \frac{80}{50} = 1.60 \text{ dollars}$$

Next, calculate the selling price with a 25% profit. A 25% profit on the cost price means the selling price is 125% of the cost price:

$$\text{Selling price per watermelon} = 1.60 \times 1.25 = 2.00 \text{ dollars}$$

2.00 dollars.

GSM8K Case 2

Question:

Regina wrote 9 novels last year. If this is 3 quarters of the number of novels she has written this year, how many novels has she written this year?

DeepSeek-R1 (772 tokens):

<think>

Okay, so Regina wrote 9 novels last year. Hmm, and that's supposed to be 3 quarters of the number she's written this year. I need to find out how many novels she wrote this year. Let me break this down.

First, let's put this into an equation. If 9 novels represent 3/4 of this year's novels, then I can set it up like this: 3/4 of this year's novels equals 9. Translating that to math, I think that would be (3/4) * x = 9, where x is the number of novels she wrote this year.

Now, to solve for x, I need to isolate it. To get rid of the fraction, I should multiply both sides by the reciprocal, which is 4/3. So multiplying both sides by 4/3 gives me x = 9 * (4/3). Let me compute that. 9 divided by 3 is 3, and then multiplied by 4 is 12. So x equals 12.

Wait, let me check if that makes sense. If she wrote 12 novels this year, then three quarters of that is indeed 9, because 12 divided by 4 is 3, and 3 times 3 is 9. Yeah, that works out. So Regina must have written 12 novels this year.

Alternatively, I can think of it this way: if 3 parts correspond to 9, then each part is 9 divided by 3, which is 3. Therefore, the whole, which is 4 parts, would be 3*4=12. That also gets me 12.

Another way to approach it could be using ratios. The ratio of last year's novels to this year's is 3:4. So setting up a proportion, 9/x = 3/4. Cross-multiplying gives 3x = 36, so x is 12 again. All the different methods seem to point to 12.

Just to ensure I haven't made any mistakes, considering that 3/4 is a fraction less than 1, so this year's total should be more than last year's 9. Since 12 is more than 9, that checks out. Which also makes sense because if 9 is three quarters, then one quarter is 3, and four quarters make 12.

No, I think that's solid. All different ways verify the answer 12. So confident about that. Let me just re-read the problem to make sure I didn't misread anything. It says Regina wrote 9 novels last year, and that this is 3 quarters of the number she wrote this year. So last year's 9 is three quarters of this year's total. Therefore, this year's total is more, so we definitely need a number more than 9, which again 12 fits. So there's no inconsistency in the answer.

So I think I'm good here. Multiply 9 by 4/3, get 12, verify with different methods, all correct. Probably 12 is the correct answer here. Makes sense.

</think>

Regina wrote 9 novels last year, which represents $\frac{3}{4}$ of her output this year. To find this year's total:

$$\frac{3}{4} \times (\text{This Year's Novels}) = 9$$

Solving for This Year's Novels:

$$\text{This Year's Novels} = 9 \times \frac{4}{3} = 12$$

****Answer:**** Regina has written 12 novels this year.

FlashThink (111 tokens):

<think>

Okay, so Regina wrote 9 novels last year. Hmm, and that's supposed to be 3 quarters of the number she's written this year. I need to find out how many novels she wrote this year. Let me break this down.

</think>

Let x be the number of novels Regina wrote this year. According to the problem, 9 novels represent 3/4 of her output this year.

This can be set up as the equation:

$$\frac{3}{4}x = 9$$

To solve for x , multiply both sides by $\frac{4}{3}$:

$$x = 9 \times \frac{4}{3} = 12$$

****Answer:**** Regina wrote **12** novels **this** year.

MATH Case 1

Question:

Evaluate $(-125)^{4/3}$.

DeepSeek-R1 (1972 tokens):

<think>

Okay, so I need to figure out what (-125) raised to the power of $4/3$ is. Hmm, exponents can be a bit tricky, especially when they're fractions and the base is negative. Alright, let me start by recalling some basic exponent rules.

First off, when you have a fractional exponent like $4/3$, that can be broken down into two parts: the denominator of the fraction is the root, and the numerator is the power. So in general, $a^{m/n}$ is the same as taking the n th root of a and then raising it to the m th power, or vice versa. Right? So, either take the root first and then the power, or take the power first and then the root. But does the order matter? I think sometimes it does, especially when dealing with negative numbers because even roots of negative numbers can be problematic in the real numbers.

Alright, so let's break down $(-125)^{4/3}$. The exponent $4/3$ can be interpreted as $4^{(1/3)}$ or $(1/3)^4$. So either take the cube root of -125 first and then raise that result to the 4th power, or raise -125 to the 4th power first and then take the cube root. But since the cube root of a negative number is defined in the real numbers, while even roots are not, maybe one path is better than the other?

Let me try both approaches and see what happens. Probably both should give the same result if done correctly.

First approach: take the cube root first, then the 4th power. The cube root of -125 . Since $(-5)^3 = -125$, so the cube root of -125 is -5 . Then raising that to the 4th power: $(-5)^4 = (-5)*(-5)*(-5)*(-5)$. Let me compute that: first, $(-5)*(-5)$ is 25, then $25*(-5)$ is -125 , and then $-125*(-5)$ is 625. So that's 625.

Second approach: raise -125 to the 4th power first, then take the cube root. So, $(-125)^4$. Let me compute that: $(-125)*(-125)$ is 15625, then $15625*(-125)$ is... hmm, let's see. Multiplying $15625*125$. Well, $15625*100$ is 1,562,500; $15625*25$ is 390,625; so $1,562,500 + 390,625 = 1,953,125$. But since it's $(-125)^4$, the sign would be positive, because any even power of a negative number is positive. So, $(-125)^4$ is 1,953,125. Then take the cube root of that. Hmm, cube root of 1,953,125. Let's see. What number cubed is 1,953,125? Let's try 10 first: 10^3 is 1000. 100^3 is 1,000,000; 15^3 is 3375, so 125^3 is 1,953,125 (to check: $125*125=15,625$; $15,625*125$ is 1,953,125). So 125^3 is 1,953,125, which means the cube root of 1,953,125 is 125. So then this approach gives 125 as well?

Wait wait, hold on. Wait, no. Wait the first approach resulted in 625 and the second in 125? That doesn't make sense. Hmm. That can't be. There must be a mistake here. The two approaches should give the same result, right?

Wait, let me check again.

First method: cube root of -125 is -5 . Then to the 4th power: $(-5)^4 = 625$.

Second method: $(-125)^4 = 1,953,125$. Then cube root of 1,953,125. Well, $1,953,125$ is $(125)^3$. Wait, but 125^3 is definitely 1,953,125. So cube root of that is 125. Wait. So this approach gave 125, but the first approach gave 625. These are different results. That's an inconsistency. So which one is correct? But wait, the answer is different depending on the order. Hmm. So maybe the fractional exponent doesn't work the same for negative bases?

Ah, now I remember. When dealing with fractional exponents where the denominator is even, we have problems with negative bases, but when the denominator is odd, we can have real roots. So since here the denominator is 3, which is odd, so in principle it's okay, but perhaps when we're applying the exponent as power first and then taking the root, we might have problems? Wait, but they both can be done in this case. So why the different answers?

Wait, perhaps this is an issue with how exponentials are defined for negative numbers with fractional exponents. Let me look back.

In real numbers, $a^{m/n}$ is defined when a is negative only if n is odd. Since if n is even, we can't take the even root of a negative number in real numbers. But here, since it's 3 (the denominator), which is odd, so it's okay. So both approaches should work. But wait, why are different answers popping up.

Wait, actually, my first approach:

First, taking the root: $(-125)^{1/3}$ is -5 . Then raising that to 4: $(-5)^4 = 625$. So that's 625.

Second approach: first raising to power 4: $(-125)^4 = (125)^4$, which is 244140625? Wait, wait, I think here's my mistake. When I computed $(-125)^4$ earlier, I must have miscalculated. Let me check that again. $(-125)^4$. Let's compute it step by step. $(-125)^2 = (-125)*(-125) = 15625$. Then, $(-125)^4 = [(125)^2]^2 = 15625^2$. Let me compute $15625*15625$. Hmm. $15,625$ times $15,625$. Let's compute this. $15,000 * 15,000 = 225,000,000$. Then $15,000 * 625 = 9,375,000$. Then $625 * 15,000 = 9,375,000$. Then $625*625=390,625$. So total would be $225,000,000 + 9,375,000 + 9,375,000 + 390,625$...

In other words:

$(15,000 + 625)*(15,000 + 625) = 15,000^2 + 2*15,000*625 + 625^2 = ?$ Which is $225,000,000 + 18,750,000 + 390,625 = 244,140,625$.

Wait, but original was just $(-125)^4$. It should be equal to $(125)^4$. Let me verify with exponentiation.

Since 125 is 5^3 . So $(5^3)^4 = 5^{12}$. 5^{12} is 244,140,625. Wait a second, $5^1=5$, $5^2=25$, $5^3=125$, $5^4=625$, $5^5=3125$, $5^6=15,625$, $5^7=78,125$, $5^8=390,625$, $5^9=1,953,125$, $5^{10}=9,765,625$, $5^{11}=48,828,125$, $5^{12}=244,140,625$.

So 5^{12} is 244,140,625. Therefore, $(125)^4 = 244,140,625$. Therefore, the correct value of $(-125)^4$ is 244,140,625, not 1,953,125. So my first calculation of $(-125)^4$ was wrong. I originally thought that $(-125)^4$ was 1,953,125, but that's actually $(125)^3$. So here's my mistake. That's confusing 3 and 4. So, in the second approach, when I calculate $(-125)^{4/3}$, doing the power first:

Raise (-125) to the 4th, which is 244,140,625. Then take the cube root. So cube root of 244,140,625 is ? Let's see, since $(5^{12})^{(1/3)} = 5^{12/3} = 5^4 = 625$. Because $5^{12} = (5^4)^3$. So $(5^{12})^{1/3} = 5^4 = 625$. So the cube root of 244,140,625 is 625. Therefore, this approach gives 625 as well, same as the first approach.

So where did I go wrong initially? I miscalculated $(-125)^4$ as $(125)^3$ instead $(125)^4$.

That was the error. So with that correction, both approaches give the same result: 625. Therefore, $(-125)^{4/3}=625$. That makes sense.

So even though the base is negative, since the denominator of the exponent is 3 (which is odd), we can still evaluate it without any issues, and the result is positive because when we take it to the 4th power, the negative becomes positive. Alternatively, if the denominator had been even, like 2/3, then there might be ambiguity because taking square roots of negative numbers is not defined in real numbers. But in this case, 4/3, the denominator is 3, which allows the cube root, which is defined for negative numbers, and then even power, resulting in a positive number.

Therefore, the final answer should be 625.

But wait, let me just cross-validate with another method. If I write $(-125)^{4/3}$, that's equivalent to $[((-125)^1]^{4/3}=[(-125)^{1/3}]^4$. Since exponents are associative in this manner, as long as the operations are defined. Given that $(-125)^{1/3}=-5$, then raising that to the 4th power gives $(-5)^4=625$.

Alternatively, the other way: $[(-125)^4]^{1/3}$. Since $(-125)^4$ is positive, then taking the cube root is fine, and $(125^4)^{1/3}=125^{4/3}=(125^{1/3})^4=(5)^4=625$.

Therefore, both ways agree.

Therefore, the answer is 625.

But just to make sure that when given a negative base and a fractional exponent, even if the denominator is odd but the numerator is even, it's okay. Another example: $(-8)^{2/3}$. Following the same principle, cube root of -8 is -2, then squared is 4. Alternatively, square first: $(-8)^2=64$, cube root of 64 is 4. Same result. So in this case, the same idea applies. If we have a negative base with exponent m/n, where n is odd and m is even, the result is positive and consistent. But if, for example, m was odd and n was odd, would that also work? Let's say $(-125)^{3/3}=(-125)^1=-125$. Also in fractions, 3/3=1, so yes. Alternatively, if n is even, and m is even, then the base must be positive. But in this case, with n odd, we can handle negative bases even if m is even or odd.

Thus, coming back to our original problem. All steps check out, so I can be confident that $(-125)^{4/3}=625$. Alright.

Final Answer $\boxed{625}$.

</think>

To evaluate $(-125)^{4/3}$, we can break down the fractional exponent into a root and a power. The exponent $\frac{4}{3}$ can be interpreted as either taking the cube root first and then raising to the 4th power, or raising to the 4th power first and then taking the cube root.

First approach:

1. Take the cube root of -125 :

$$\sqrt[3]{-125} = -5$$

2. Raise the result to the 4th power:

$$(-5)^4 = (-5) \times (-5) \times (-5) \times (-5) = 625$$

Second approach:

1. Raise -125 to the 4th power:

$$(-125)^4 = 125^4 = (5^3)^4 = 5^{12} = 244140625$$

2. Take the cube root of the result:

$$\sqrt[3]{244140625} = 5^4 = 625$$

Both approaches yield the same result. Therefore, the value of $(-125)^{4/3}$ is

$$\boxed{625}.$$

FlashThink (150 tokens):

<think>

Okay, so I need to figure out what (-125) raised to the power of $4/3$ is. Hmm, exponents can be a bit tricky, especially when they're fractions and the base is negative. Alright, let me start by recalling some basic exponent rules.

</think>

The expression $(-125)^{4/3}$ can be evaluated by breaking it down into two steps:

1. **Finding the cube root (denominator of the exponent):**

$$(-125)^{1/3} = -5$$

because $(-5)^3 = -125$.

2. **Raising the result to the 4th power (numerator of the exponent):**

$$(-5)^4 = 625.$$

Combining these steps:

$$(-125)^{4/3} = \left((-125)^{1/3} \right)^4 = (-5)^4 = 625$$

Answer: $\boxed{625}$

inequality is $6 - x > -9$. Alright, so I have to solve for x , right? Then once I find the solution set, the greatest integer would be the largest whole number that's still part of that solution.

First, maybe I should isolate x . Let me remember how to solve inequalities. The same way as equations, but if I multiply or divide by a negative number, the inequality sign flips. Right. So starting with $6 - x > -9$. Let me subtract 6 from both sides. That cancels out the 6 on the left. So, $6 - x$ minus 6 would be $-x > -9 - 6$. Calculating that, $-9 - 6$ is -15 . So now we have $-x > -15$.

Next step, to solve for x , I need to get rid of that negative sign in front of x . So I multiply both sides by -1 . But wait, multiplying both sides of an inequality by a negative number reverses the inequality sign. So, if I multiply both sides by -1 , right, then I get $x < 15$. Let me check that again. Original inequality after subtracting 6: $-x > -15$. Multiply both sides by -1 : $x < 15$. Yup, that's correct. The direction of the inequality flips when multiplying by a negative.

So, the solution is all real numbers x such that x is less than 15. Now, the question asks for the greatest integer that satisfies this inequality. Since x has to be less than 15, the greatest integer less than 15 would be 14. Wait a second, is 14 the largest integer that's still not 15? But 15 is the boundary here. Wait, but since x has to be less than 15, we can't include 15. So integers go ..., 12, 13, 14, 15, 16... but x has to be less than 15, so the maximum integer is 14.

But wait, let me test with actual numbers to ensure I haven't made a mistake. Let me substitute $x = 14$ into the original inequality: $6 - 14 = -8$. Which is greater than -9 ? Yes, because -8 is to the right of -9 on the number line. Okay, that works. Now, what if we tried $x = 15$? Then $6 - 15 = -9$. The original inequality says $6 - x$ must be greater than -9 . So, -9 is not greater than -9 , so 15 is not a solution. So, yes, 14 is the highest integer that works. But maybe I should check another number just to be thorough. Let's try $x = 14.5$. That's a non-integer. $6 - 14.5 = -8.5$, which is still greater than -9 , so that number is a solution. But since the question asks for the greatest integer, integers are whole numbers, so 14.5 isn't an integer. So the next one down is 14, which checks out.

Is there a case where maybe another number could satisfy this but potentially higher? Like 14.9? $6 - 14.9 = -8.9$, which is still greater than -9 . Hmm, so even numbers approaching 15 from below, like 14.999, would result in $6 - 14.999 = -8.999$, which is still greater than -9 . So really, as long as x is less than 15, it's a solution. But again, integers specifically. So greatest integer less than 15 is 14. Let me just make sure we didn't flip the inequality incorrectly. So when we had $-x > -15$. When we divide or multiply by a negative, the inequality flips. So dividing both sides by -1 , $x < 15$. That part seems correct.

Alternatively, maybe solving it another way just to verify. Let me add x to both sides first. Starting again: $6 - x > -9$. If I add x to both sides, the inequality becomes $6 > -9 + x$. Then, to isolate x , I'd add 9 to both sides: $6 + 9 > x$, so $15 > x$, which is the same as $x < 15$. Yup, same result. So same answer, which confirms the earlier solution.

Therefore, the greatest integer is indeed 14. Hmm, solid. Unless there's a trick here that I'm missing. But since the inequality is strict (doesn't include equality), 15 is excluded, so 14 is the answer.

Final Answer

The greatest integer that satisfies the inequality is $\boxed{14}$.

</think>

To solve the inequality $6 - x > -9$ and find the greatest integer that satisfies it, we start by isolating x .

First, subtract 6 from both sides:

$$6 - x - 6 > -9 - 6$$

This simplifies to:

$$-x > -15$$

Next, multiply both sides by -1, remembering to reverse the inequality sign:

$$x < 15$$

The solution set includes all real numbers less than 15. The greatest integer less than 15 is 14. We verify this by substituting $x = 14$ into the original inequality:

$$6 - 14 = -8$$

Since -8 is greater than -9 , $x = 14$ is a solution. Testing $x = 15$ results in:

$$6 - 15 = -9$$

Which does not satisfy the inequality as -9 is not greater than -9 .

Thus, the greatest integer that satisfies the inequality is $\boxed{14}$.

FlashThink (155 tokens):

<think>

Okay, let's see. I need to find the greatest integer that satisfies the inequality $6 - x > -9$. Hmm, let me write that down first. The inequality is $6 - x > -9$. Alright, so I have to solve for x , right? Then once I find the solution set, the greatest integer would be the largest whole number that's still part of that solution.

</think>

To solve the inequality $6 - x > -9$, follow these steps:

1. **Subtract 6 from both sides** to begin isolating x :

$$6 - x - 6 > -9 - 6$$

Simplifying this gives:

$$-x > -15$$

2. **Multiply both sides by -1** to solve for x . Remember that multiplying or dividing an inequality by a negative number reverses the inequality sign:

$$x < 15$$

3. **Identify the greatest integer** that is less than 15. The largest whole number less than 15 is 14.

Answer: $\boxed{14}$

MATH Case 2

Question:

What is the greatest integer that satisfies $6 - x > -9$?

DeepSeek-R1 (1256 tokens):

<think>

Okay, let's see. I need to find the greatest integer that satisfies the inequality $6 - x > -9$. Hmm, let me write that down first. The

GPQA Diamond Case 1

Question:

Two astronauts of masses $2m$ and $3m$ travel at relativistic speeds of $0.6c$ and $0.5c$, respectively, in the positive x -direction. Here, c is the speed of light. What will be the relative speed and the total energy of the system?

- A) $v_{\text{rel}} = 1.1c$, $E = mc^2$
 B) $v_{\text{rel}} = 0.14c$, $E = 5mc^2$
 C) $v_{\text{rel}} = 0.14c$, $E = 5.96mc^2$
 D) $v_{\text{rel}} = 0.1c$, $E = 4.96mc^2$

DeepSeek-R1 (1972 tokens):

<think>

Okay, let's see. I need to find the relative speed between the two astronauts and the total energy of the system. The problem says they're moving along the positive x -direction, so I guess their speeds are all in the same direction. But wait, since they're moving at relativistic speeds, I can't just add their velocities the classical way, right? Wait, relative velocity. Oh right, in special relativity, velocity addition isn't straightforward. So their relative speed isn't just $0.6c - 0.5c = 0.1c$. That would be classical thinking, but in reality, we need to use the Lorentz velocity addition formula. But wait, velocity addition is when you have a frame moving with a certain velocity. Let me think. Astronaut A is moving at $0.6c$ in the lab frame (assuming lab frame is what we're measuring everything in). Astronaut B is moving at $0.5c$ in the same frame. So from the perspective of astronaut A, the lab is moving at $-0.6c$. What's the velocity of B from A's frame? The formula for the velocity observed by a moving observer (A) for object B? The relativistic addition formula. The formula is $u' = \frac{u-v}{1-\frac{uv}{c^2}}$. So the

frame of the lab finds astronaut A moving at $v = 0.6c$. Then in astronaut A's frame, astronaut B's speed would be $\frac{0.5c-0.6c}{1-\frac{0.5 \times 0.6c^2}{c^2}}$. Let me compute that.

The numerator is $-0.1c$. The denominator is $1 - 0.3$ (because 0.5×0.6 is 0.3 , and c^2 cancels out). So denominator is 0.7 . So $u' = \frac{-0.1c}{0.7}$ approximately $-0.1428c$. But then the negative sign just indicates direction relative to A's frame, right? Wait wait, but astronaut B is moving at $0.5c$ in the lab's frame (same direction as A, since both in $+x$). Wait that would make sense. From A's frame, B is approaching from behind? Because A is moving faster ($0.6c$ compared to B's $0.5c$), so from A's viewpoint, B is moving in the opposite direction. Wait but their relative speed would be the magnitude, but this calculation gives u' as roughly $-0.1428c$, which is like moving backwards at $0.14c$. So the relative speed would be the magnitude, which is about $0.14c$. Hmm. So in the options, answer B and C have $0.14c$ for v_{rel} . So answer's likely B or C.

Now what about the total energy? The total energy is the sum of their relativistic energies. The formula for relativistic energy is γmc^2 . So each astronaut's gamma is calculated first. For the astronaut with mass $2m$ moving at $0.6c$. Gamma is $\frac{1}{\sqrt{1-\frac{v^2}{c^2}}}$.

Calculating gamma for $0.6c$: $\frac{v^2}{c^2} = 0.36$. So $1 - 0.36 = 0.64$, $\sqrt{0.64} = 0.8$. So gamma is $\frac{1}{0.8} = 1.25$. So energy $E_A = \text{gamma} * m_{\text{rest}} * c^2 = 1.25 * 2mc^2 = 2.5mc^2$.

For the other astronaut (mass $3m$, speed $0.5c$): velocity squared is $0.25c^2$. Gamma for $0.5c$: $\sqrt{1-0.25} = \sqrt{0.75} \approx 0.866$. So $\frac{1}{0.866} \approx 1.1547$. Gamma is 1.1547 . So $E_B = 1.1547 * 3mc^2 \approx 3.4641mc^2$. Adding them together: $2.5 + 3.4641 = 5.9641$, around $5.96mc^2$. So the total energy is approximately $5.96mc^2$. That would correspond to option C: relative speed is approx $0.14c$ (as in answer C), and total E 5.96 .

Option C lists $v_{\text{rel}} = 0.14c$, and $E = 5.96mc^2$. So the correct answer would be C. The other options... B says $E = 5mc^2$ which would be too low. Checking the total energy from each astronaut again. $2m * \text{gamma}_1$ and $3m * \text{gamma}_2$.

Recomputing the gamma. Let's be exact.

Original velocity M1: $0.6c$. $v^2 = 0.36c^2$. So $\sqrt{1-0.36} = \sqrt{0.64} = 0.8$. So gamma is $\frac{1}{0.8} = \frac{5}{4} = 1.25$. E1: $2m * \frac{5}{4}c^2 = \frac{10}{4}mc^2 = 2.5mc^2$. Correct.

M2 velocity $0.5c$. $v^2 = 0.25c^2$. $1 - 0.25 = 0.75$. Square root is $\sqrt{0.75} = \sqrt{\frac{3}{4}} = \frac{\sqrt{3}}{2} \approx 0.866$. So gamma is $\frac{1}{\frac{\sqrt{3}}{2}} = \frac{2}{\sqrt{3}}$? Wait

alternative way: $\frac{1}{\sqrt{0.75}} = \frac{2}{\sqrt{3}} \approx 1.1547$. So yeah, gamma is 1.1547 .

So E2: $3m * (1.1547)c^2 \approx 3.464mc^2$. Add $2.5 + 3.464 \approx 5.964$. So yes, $5.96mc^2$ for E. Choice C.

So the answers would be $v_{\text{rel}} = 0.14c$ (from the prior calculation), and $E \approx 5.96mc^2$. The correct option is C.

</think>

To solve the problem, we analyze the relative speed and total relativistic energy of the two astronauts.

****Relative Speed****

The astronauts move in the same direction in the lab frame: Astronaut A at $0.6c$, and Astronaut B at $0.5c$. To find the relative speed of B with respect to A, we use the relativistic velocity addition formula:

$$u' = \frac{u - v}{1 - \frac{uv}{c^2}},$$

where $v = 0.6c$ (A's speed relative to the lab) and $u = 0.5c$ (B's speed relative to the lab). Substituting:

$$u' = \frac{0.5c - 0.6c}{1 - \frac{(0.5c)(0.6c)}{c^2}} = \frac{-0.1c}{1 - 0.3} = \frac{-0.1c}{0.7} \approx -0.1428c.$$

The negative sign indicates B is moving in the opposite direction relative to A's frame. The ****magnitude**** of the relative speed is $0.14c$.

****Total Energy****

The total relativistic energy of the system is the sum of the individual energies.

Relativistic energy is given by $E = \gamma mc^2$, where $\gamma = \frac{1}{\sqrt{1-v^2/c^2}}$.

- Astronaut A (mass $2m$, speed $0.6c$):

$$\gamma_A = \frac{1}{\sqrt{1 - (0.6c)^2/c^2}} = \frac{1}{\sqrt{0.64}} = \frac{1}{0.8} = 1.25.$$

$$E_A = \gamma_A \cdot 2m \cdot c^2 = 1.25 \cdot 2m \cdot c^2 = 2.5mc^2.$$

- Astronaut B (mass $3m$, speed $0.5c$):

$$\gamma_B = \frac{1}{\sqrt{1 - (0.5c)^2/c^2}} = \frac{1}{\sqrt{0.75}} = \frac{2}{\sqrt{3}} \approx 1.1547.$$

$$E_B = \gamma_B \cdot 3m \cdot c^2 \approx 1.1547 \cdot 3m \cdot c^2 = 3.464mc^2.$$

- Total Energy:

$$E_{\text{total}} = E_A + E_B = 2.5mc^2 + 3.464mc^2 \approx 5.96mc^2.$$

****Final Answer****

The relative speed is approximately $0.14c$, and the total energy is approximately $5.96mc^2$. Both values match ****Option C****.

ANSWER: C

FlashThink (639 tokens):

<think>

Okay, let's see. I need to find the relative speed between the two astronauts and the total energy of the system. The problem says they're moving along the positive x -direction, so I guess their speeds are all in the same direction. But wait, since they're moving at relativistic speeds, I can't just add their velocities the classical way, right?

</think>

To solve the problem, we start by addressing the two components: the relative speed between the astronauts and the total energy of the system.

****Relative Speed****

In relativity, velocities do not add linearly. Instead, the relative velocity v_{rel} between two objects moving at speeds u and v in the same direction is given by the relativistic velocity addition formula:

$$v_{\text{rel}} = \frac{u - v}{1 - \frac{uv}{c^2}}.$$

Here, $u = 0.6c$ and $v = 0.5c$. Substituting:

$$v_{\text{rel}} = \frac{0.6c - 0.5c}{1 - (0.6)(0.5)} = \frac{0.1c}{1 - 0.3} = \frac{0.1c}{0.7} \approx 0.14c.$$

****Total Energy****

The total relativistic energy of the system is the sum of the individual relativistic energies:

$$E_{\text{total}} = \gamma_1 m_1 c^2 + \gamma_2 m_2 c^2,$$

where $\gamma = \frac{1}{\sqrt{1-v^2/c^2}}$.

1. For the first astronaut ($m_1 = 2m$, $v_1 = 0.6c$):

$$\gamma_1 = \frac{1}{\sqrt{1 - (0.6)^2}} = \frac{1}{\sqrt{0.64}} = \frac{1}{0.8} = 1.25,$$

$$E_1 = 1.25 \cdot 2mc^2 = 2.5mc^2.$$

2. For the second astronaut ($m_2 = 3m$, $v_2 = 0.5c$):

$$\gamma_2 = \frac{1}{\sqrt{1 - (0.5)^2}} = \frac{1}{\sqrt{0.75}} \approx 1.1547,$$

$$E_2 \approx 1.1547 \cdot 3mc^2 \approx 3.464mc^2.$$

Adding the energies:

$$E_{\text{total}} \approx 2.5mc^2 + 3.464mc^2 = 5.964mc^2.$$

****Verifying the Options****

- ****Option B**** claims $E = 5mc^2$, which is incorrect because the calculation gives approximately $5.96mc^2$.

- ****Option C**** matches both $v_{\text{rel}} \approx 0.14c$ and $E \approx 5.96mc^2$, making it the correct answer.

- Options A and D are inconsistent with the relativistic calculations.

ANSWER: C

B Verification model prompts

This paper uses LLMs as the verification models, and uses prompt engineering to let the verification model output yes or no to perform a binary classification task, which serves as a signal for whether to end the reasoning phase early. The prompt used is shown below.

Qwen-2.5

<lim_start>system

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

<lim_endl>

<lim_start>user

Based on the following question and thought, please judge whether the thought is sufficient to support solving the question. Please directly output yes or no instead of outputting other content.

Question

{question}

Thought

{thought}

<lim_endl>

<im_start>assistant

Llama-3.1

<lbegin_of_textl><lstart_header_idl>system<lend_header_idl>

Based on the following question and thought, please judge whether the thought is sufficient to support solving the question. Please directly output yes or no instead of outputting other content.

Question

{question}

Thought

{thought}

<leot_idl><lstart_header_idl>assistant<lend_header_idl>

Mistral

<s>[INST]

Based on the following question and thought, please judge whether the thought is sufficient to support solving the question. Please directly output yes or no instead of outputting other content.

Question

{question}

Thought

{thought}

[/INST]

Qwen-3

<lim_start>user

Based on the following question and thought, please judge whether the thought is sufficient to support solving the question. Please directly output yes or no instead of outputting other content.

Question

{question}

Thought

{thought}

<lim_endl>

<lim_start>assistant